

かんすう 関数

やさしい日本語で学ぶ JavaScript の「関数」ガイド

皆さん、こんにちは。JavaScriptを勉強中の留学生の皆さんにとって、一番大切な道具（どうぐ）のひとつが「関数」です。関数を使いこなせると、プログラムを整理（せいり）して、賢（かしこ）く書くことができます。

実は、今までにも、関数は使っていました。alert(), Number()などは、あらかじめJavaScriptが用意している関数です。

一緒に学んでいきましょう。

I. 用語の整理

関数について説明する前に、整理しておきたい言葉があります。前に出てきた言葉ですが、ここで復習しておきましょう。

プログラミングを勉強していると、漢字が並んでいて難しく感じますよね。「演算子」と「式」と「文」です。これらは、プログラミングの「文法（ぶんぽう）」の名前です。そして、どれもプログラムの動きを書くためのものです。

日本語の「単語（たんご）」「句（く）」「文（ぶん）」の違いに似ています。

わかりやすく「料理」に例えて説明しますね。

演算子（えんざんし）／ Operator

→ 「道具（どうぐ）」のこと

数字を足したり、比べたりするための「記号（きごう）」です。「料理の道具」のようなものです。

- 例：+, -, *, /, =, >, &&
- 料理でいうと：包丁（ほうちょう）、お鍋（なべ）、フライパン

しかし、道具を用意しただけでは何も起きません。道具を使って何かをする必要があります。

式（しき）／ Expression

→ 「計算のフレーズ」のこと

演算子（道具）を使って、「答え（値）」を出すものです。

- 例：10 + 5（答えは15）、a === b（答えは「はい」か「いいえ」）
- 料理でいうと：「野菜（やさい）を切る」「お肉を焼く」
- 何かのアクションをして、「切った野菜」という結果が生まれます。

文（ぶん）／ Statement

→「コンピューターへの^{めいれい}命令」のこと

「～をしてください」という、ひとつの^{かんぺき}完璧な指示です。

JavaScriptでは、最後にセミコロン ; をつけます。

- 例：^{れい}let x = 10 + 5; （xという箱に、15を入れてください！）
- 料理でいうと：「カレーを作ってください！」
- 道具（^{えんざん}演算子）を使い、ステップ（^く式）を組み合わせて、ひとつの^あ仕事（文）を終わらせます。

まとめの比較表

名前	英語	イメージ	役割（やくわり）
^{えんざん} 演算子	Operator	道具	+ や = などの記号
式	Expression	フレーズ	何かの「 ^ち 値（あたい）」になるもの
文	Statement	命令（文）	コンピューターに「 ^{どうさく} 動作」をさせるもの

[!TIP] ^{みわ}見分けかたのコツ：最後に ; （セミコロン）がついているものは、ほとんどの場合「文」です。日本語の「。」（まる）と同じです。

II. 関数とは何か？

「^{かんすう}関数」は、プログラミングの中でとても便利な「^{まほう}魔法の箱」や「^{じどうはんばいき}自動販売機」のようなものです。

漢字で見ると難しそうですが、その中身は、いくつかの「文」をひとつ（ひとつ）にまとめた「^{むずか}セット」です。とてもシンプルですよ！

^{かんすう}関数は「料理マシン」と同じ！

関数をイメージするときは、キッチンにある「ジューサー（ミキサー）」を思い出してください。

1. 入れるもの（^{ひきすう}引数：ひきすう）：リンゴを入れる。
2. 中での仕事（^{しゅり}処理：しゅり）：リンゴを細かくつぶす。
3. 出てくるもの（^{もどち}戻り値：もどりち）：リンゴジュースが出てくる！

一度この「マシン」を作っておけば、次からは「リンゴ」を入れるだけで、いつでもリンゴジュースを作れます。それだけでなく、「オレンジ」や「バナナ」を入れれば、オレンジジュースやバナナジュースを作ることのできるのです。

こんどは、「自動販売機（^{じどうはんばいき}）」に例えて見ます。

- お金を入れる・ボタンを押す：これが、関数に渡すデータ「^{ひきすう}引数（ひきすう）」です。
- 飲み物が出る：これが、命令を実行した結果である「^{もどち}戻り値（もどりち）」です。

自動販売機の中には、「お金を数える」「在庫（ざいこ）を確認する」といった複雑（ふくざつ）なプログラムが入っています。でも、私たちは「ボタンを押す」というひとつの命令だけで、欲しい結果を受け取ることができます。

なぜ「関数」を使うの？

同じコードを何度も書くのは大変だし、間違いのもとです。関数を使うメリットは2つあります。

- **楽ができます**：一度作れば、名前を呼ぶだけで何度でも使えます。同じことを何度も書かなくてよくなります。
- **直しやすくなります**：挨拶の言葉を変えたいとき、1ヶ所（関数の中）を直すだけで全部変わる。
- **整理しやすくなります**：長いコードがスッキリします。

次は、この便利な「関数」をどうやって作るのか、ルールを見ていきましょう。

III. 関数の作り方（定義：ていぎ）

関数を新しく作ることを「定義（ていぎ）」と言います。JavaScriptでは、誰が見ても分かりやすいように、名前の付け方に世界共通のルールがあります。

基本の書き方

以下のキーワードと記号を組み合わせて作ります。

- **function**：「今から関数を作ります」という合図（あいず）です。
- **関数名**：そのセットに付ける名前です。
- **()**：引数（データ）を受け取るための場所です。
- **{}**：この中に、実行したい命令をすべて書きます。これを「ブロック」と呼びます。

良い名前をつけるルール

プログラマーは、関数が「何をするものか」すぐ分かるように、以下のルールを守ります。

1. 動詞（どうし）から始める：get（取る）、show（見せる）、calculate（計算する）、handle（処理する）などの動詞を使います。
2. キャメルケース（camelCase）：2つ目以降の単語の最初を大文字にします。例：showMessage, calculateTotal

コードの例

「挨拶（あいさつ）をしてくれるマシン」を作ってみましょう。

```
// 準備: 関数 (かんすう) を定義 (ていぎ) する
function sayHello() {
  console.log("こんにちは！JavaScriptの世界へようこそ。");
}
```

これで「準備」ができました。次は、これを目的に合わせて「動かす」方法を説明します。

IV. 関数の呼び出し (実行: じっこう)

関数を作っただけでは、プログラムは動きません。作った関数を動かすことを「呼び出し」と言います。初心者の方は「定義 (準備)」と「呼び出し (実行)」を分けて考えることがとても大切です。

呼び出しの仕組み

関数名のあとに () をつけると、中身が動き出します。sayHello(); と書くと、先ほど {} の中に書いたメッセージが表示されます。

イベントとの連携 (れんけい)

ウェブページでは、ボタンをクリックした時に関数を呼び出すことが多いです。これを「イベントハンドラ」と言います。

HTMLとJavaScriptのつながり

[HTML側]

```
<button onclick="sayHello()">ボタンを押してね</button>
```

↓

(クリックされると、関数 (かんすう) を呼び出す)

↓

[JavaScript側]

```
function sayHello() {
  alert("こんにちは！");
}
```

V. 引数 (ひきすう) の活用と注意点

今度は^{ひきすう}引数をとる^{かんすう}関数を作ってみましょう。

「名前を呼んで、挨拶（あいさつ）をしてくれるマシン」を作ってみましょう。

```
// 1. 定義（ていぎ）：マシンを作る
function sayHello(name) {
  return "こんにちは、" + name + "さん！";
}
```

- **function**：「これからマシンを作るよ！」という合図。
- **name** (^{ひきすう}引数)：マシンに入れる材料（ここでは名前）。
- **return** (^{もと}戻り^ち値)：マシンが最後に出してくれる結果。

関数に^{かんすう}**name**という^{へんすう}変数に入れたデータを渡すことで、処理を柔軟（じゅうなん）^{じゅうなん}（じゅうなん）に変えることができます。

では、^{ひきすう}引数をとる^{かんすう}関数を呼び出すときは、どのようにすればよいのでしょうか？

```
// 2. 実行：ボタンを押して動かす
let message = sayHello("田中");
console.log(message); // 「こんにちは、田中さん！」と出る
```

このように、**()**の中に^{かんすう}関数に渡すデータを書きます。

ここで、

- ^{ひきすう}仮引数（かりひきすう）：^{かんすう}関数の中で使う「データの箱」の名前。ここでは**name**
- ^{ひきすう}実引数（じつひきすう）：呼び出すときに実際に渡す「中身」のデータ。ここでは"田中"という文字列

というふうに呼びます。この呼び方もよく使うので、覚えてください。

次は、^{へんすう}変数の「使える範囲」について学びます。

^{へんすう}VI. 変数のスコープ（Scope）：^{へんすう}変数の見える範囲

^{へんすう}変数は、どこからでも見えるわけではありません。この「見える範囲」をスコープと呼びます。スコープを正しく使うことで、プログラムのミス（バグ）を減らすことができます。

プログラミングの「スコープ（Scope）」は、日本語でいうと「有効（ゆうこう）な範囲（はんい）」、つまり「^{へんすう}その変数がどこまで使えるか」というルールのことです。

グローバルスコープ (Global Scope)

→ 「外の世界（公園やインターネット）」のようなもの

プログラムのどこからでも見える、一番広いスコープです。

```
let world = "こんにちは世界!"; // グローバル変数（へんすう）

function sayWorld() {
  console.log(world); // 関数（かんすう）の中でも使える
}

sayWorld();
console.log(world); // 関数（かんすう）の外でも使える
```

- **イメージ**：空に浮かんでいる「お月様（つきさま）」です。どこにいても見えますよね。

ローカルスコープ / 関数スコープ (Local Scope)

→ 「家の中」のようなもの

function (関数) の中で作った変数は、その関数の外からは見えません。

```
function myHouse() {
  let secret = "これは秘密（ひみつ）です"; // ローカル変数（へんすう）
  console.log(secret); // 関数（かんすう）の中ならOK
}

myHouse();
console.log(secret); // ✖ エラー！外からは見えません。
```

- **イメージ**：家の中にある「テレビのりもこん」です。隣の家の人（関数の外）は使えません。

ブロックスコープ (Block Scope)

→ 「小さな箱（はこ）」のようなもの

if 文や **for** 文などの **{ }**（波（なみ）カッコ）の中で **let** や **const** を使って作った変数は、その **{ }** の中だけでしか使えません。

```
if (true) {
  let fruit = "りんご"; // ブロック変数（へんすう）
  console.log(fruit); // ブロックの中ならOK
}

console.log(fruit); // ✖ エラー！箱の外には出せません。
```

- **イメージ**：ロッカーの中の荷物です。ロッカー（**{ }**）を閉めたら、外からは触れません。

ポイントまとめ

スコープの名前	使える場所	わかりやすい例え ^{たと}
グローバル	どこでもOK	公園にある時計（みんな見れる）
ローカル（関数） ^{かんすう}	その関数の中だけ ^{かんすう}	あなたの部屋の電気（外からは消せない）
ブロック	{ } の中だけ	お弁当箱の中のおかず（外にはこぼれない）

なぜこれが大事なの？

もし、すべての変数^{へんすう}を「グローバル（どこでも使える）」にすると、知らないうちに別の場所でデータ^{うわが}が上書き（書きかえ）されてしまい、バグの原因になります。

[!TIP] 「できるだけ狭（せま）く」が基本！ 留学生には、「変数^{へんすう}はなるべく小さな箱（ブロックスコープ）に入れて、必要なときだけ外に出そうね」と教えてあげてください。

var は使わない、const を使おう

JavaScriptには変数^{へんすう}の作り方が3つありますが、安全性のために使い分けが重要です。

var・let・const の違いは、プログラミングの世界では「古いルール」と「新しいルール」の違いだと考えると分かりやすいです。

むかしは var しかありませんでしたが、今はもっと安全な let と const を使うのが普通です。

違いをまとめた表

まずは、この表で全体（ぜんたい）のルールを見てみましょう。

種類	スコープ（範囲）	値 ^か の書き換え ^{かえ}	同じ名前で作り直し
var	関数 ^{かんすう} スコープ（広い）	OK	OK（危ない！）
let	ブロック スコープ（狭い）	OK	✖ だめ
const	ブロック スコープ（狭い）	✖ だめ	✖ だめ

なぜ var は使わないほうがいいのか？

var は「ルールがゆるすぎる」ので、ミスが起きやすいんです。

① ブロック { } を無視して外に出てしまう

var は if や for の外まで飛び出してしまう。

```
if (true) {
  var leak = "外に出ちゃうよ！";
}
console.log(leak); // ○ 表示（ひょうじ）できてしまう（これがバグの原因になる）
```

```
if (true) {
  let safe = "外には出ないよ";
}
console.log(safe); // ✖ エラー（外からは見えないので安全！）
```

② 同じ名前で何度でも作れてしまう

var は、同じ名前の変数^{へんすう}をうっかり2回作っても怒られません。

```
var name = "田中";
var name = "鈴木"; // ○ OK（前の「田中」が消えてしまう！）

let age = 20;
let age = 25; // ✖ エラー！「もうありますよ！」と教えてくれる
```

let と const はどう使い分ける？

今のプログラミングでは、この「^{つか}^わ使い分け」がとても大切です。^く^{かえ}繰り返しになりますが、ここでも確認しておきましょう。

const（定数：ていすう）

- 「カギをかけた箱」です。
- 一度入れたら、中身を変えることはできません。
- 基本はこれを使います。途中で値が変わらないほうが、読む人が安心できるからです。

let（変数：へんすう）

- 「カギのない箱」です。
- 中身を何度も入れ替^い^{かえ}ることができます。
- **for** 文のカウンター（**i++** など）のように、**数字が変わるものだけ**に使います。

学生へのアドバイス

1. まずは **const** を使おう！（一番安全だから）
2. もし、あとで値を書き換^か^{かえ}える必要があるなら **let** に変えよう。
3. **var** はもう使わない！（古い本や古いサイトには書いてあるけど、今は使わないのがルールだよ）

[!TIP] 「定数（const）」の「定」は「決定（けってい）」の「定」。つまり、「もう決まっていて動かない」という意味だと教えると分かりやすいですよ。

VII. 無名関数（むめいかんすう）

ここまで「関数（マシン）」についてお話ししました。次は、そのマシンの「ちょっと特別な形」である、無名関数（むめいかんすう）とアロー関数について説明します。

どちらも、今のJavaScriptでは毎日使うとても大切な書き方です！

無名関数（むめいかんすう）

→「名前のないマシン」のこと

普通の関数は `function sayHello() ...` のように名前をつけますが、名前をつけないで作ることもできます。

- **なぜ名前をつけないの？** 「その場ですぐに使うだけだから、名前をつけるのが面倒（めんどろ）」という時に使います。
- **例え話：** 名前のある関数が「名前を登録（とうろく）した専属のシェフ」なら、無名関数は「その時だけ手伝ってくれる助っ人（すけっと）」のようなイメージです。

```
// 普通の関数（かんすう）（名前がある）
function sayHi() {
  console.log("こんにちは！");
}

// 無名関数（かんすう）（名前がない）
// 変数（へんすう）に入れて使うことが多いです
const sayHi2 = function() {
  console.log("こんにちは！");
};
```

アロー関数（Arrow Function）

→「もっと短く書ける、最新のショートカット」のこと

「無名関数」をさらに短く、カッコよく書く方法です。=>（矢印：アロー）を使うので「アロー関数」と呼びます。

書き方の変化を見てみよう

どれも同じ動きをしますが、下に行くほど短くなります。

1. 普通の関数： `function(a, b) { return a + b; }`
2. アロー関数： `(a, b) => { return a + b; }`
3. もっと短く（1行のとき）： `(a, b) => a + b;`

どうしてアロー関数を使うの？

留学生のみなさんには、この3つのメリットを覚えてほしいです。

- **タイピングが楽**： `function` と書かなくていいので、指が疲れません。
- **見た目がスッキリ**： コードが短くなって、読みやすくなります。
- **「this」のルールがやさしい**：（少し難しい話ですが）アロー関数を使うと、プログラムが「自分（this）」を見失いにくくなり、エラーが減ります。

どんな時に使うの？（具体的な例）

一番よく使うのは、「他の関数の中で、ちょっと手伝ってほしい時」です。

```
// 3秒後に「時間ですよ！」と表示（ひょうじ）する
setTimeout(() => {
  console.log("時間ですよ！");
}, 3000);
```

このように、`setTimeout` という関数の中に、さらに小さな関数（アロー関数）を入れて使います。

学生へのアドバイス

名前	書き方の特徴	ニュアンス
普通の関数	<code>function 名前() { ... }</code>	どこでも使える、基本の形
無名関数	<code>function() { ... }</code>	名前をつけるのが面倒なとき
アロー関数	<code>() => { ... }</code>	今、一番よく使う形！ かっこよくて便利

[!TIP] 初心者のうちは... 自分で書くときは、まず「アロー関数」を練習してみてください。今の現場では90% くらいアロー関数が使われています。もし古いコードを読んで `function` が出てきたら、「あ、これはお父さん世代の書き方だな」と思えばOKです！

いかがでしょうか？「矢印（=>）が出てきたら、短い関数の合図だ！」と覚えると簡単ですよ。

VIII. まとめ

JavaScriptの「関数」をマスターするために、5つのポイントを振り返りましょう。

- 定義： `function` を使い、動詞から始まる名前をつけてセットにしましょう。
- 呼び出し： 関数名のあとの `()` を忘れずにつけて実行しましょう。
- スコープ： 変数は `const` を使い、なるべく `{ }` の中で安全に管理しましょう。
- 無名関数： 一度きりの処理は、名前をつけずにスマートに書きましょう。

So What?（だから何？）^{かんすう}関数を学ぶことは、ただコードを書くことではありません。大きな問題（プログラム）を、小さくて扱いやすい「^{かんすう}関数のセット」に分けて考えることです。これが「プログラマーのように考える」ための^{いっぽ}第一歩です。

留学生向けの言葉のガイド

^{むずか}難しい専門用語を、もっと簡単な日本語に言いかえてみましょう。

専門用語	やさしい言い換え	英語
^{ていぎ} 定義（ていぎ）する	マシンを作る	Define
呼び出す / 実行する	マシンを動かす	Call / Execute
^{ひきすう} 引数（ひきすう）	入れるもの / 材料	Arguments / Parameters
^{もと} ^ち 戻り値（もどりち）	返ってくるもの / 答え	Return value

[!TIP] ^{ひきすう}「引数（ひきすう）」の覚え方 「いんすう」と読みたくなりますが、プログラミングでは「ひきすう」と読みます。「(材料を) 引き連れてくる数」だと覚えるといいですよ！